

Smart Municipal Assistance System - API Architecture v1.0

1. Purpose

This API architecture defines the Node.js/Express backend contract for the AI-powered smart municipal platform. It is aligned with the MongoDB database architecture v4.0 AI Extension and preserves the database decision that existing operational collections remain authoritative while AI, public tracking, reporting, and governance features are added through explicit service boundaries.

The API must not allow frontend clients, Gemini, OCR, face-recognition services, or any external integration to write directly to MongoDB. All writes pass through validated backend services, authorization checks, audit logging, and collection-specific rules.

2. Database Alignment Summary

The database architecture contains 42 collections: 26 preserved production collections and 16 added AI/frontend/SRS collections. The previous API folder only documented expected deliverables, so the API architecture needed to be expanded to cover the v4.0 database contract.

Database area	Required API architecture change
users, roles, permissions, auth_sessions, password_reset_tokens	Add Auth, Session, RBAC, and Password Reset modules with token hashing, audit logging, and account/session controls.
complaints, complaint_assignments, complaint_status_history, sla_policies, feedback	Add Complaint, Assignment, Timeline, SLA, and Feedback APIs with citizen ownership, staff department scope, and admin assignment controls.
file_tracking, file_tracking_history	Add public file tracker and staff/admin file update APIs; public responses must expose only public fields and public_note.
knowledge_base, kb_embeddings, chatbot_sessions, chatbot_messages	Add AI Gateway and Knowledge Base APIs for Gemini/RAG, approved retrieval, redacted chat logs, prompt metadata, and six-month retention fields.
ai_human_review_queue, notifications	Add Human Review and Notification APIs for chatbot escalation, low-confidence biometric review, failed liveness, and AI exceptions.
attendance, attendance_summary, biometric_consent, face_templates, staff_presence	Add Attendance and Biometric APIs with active consent checks, encrypted template references, liveness/confidence validation, manual fallback, and presence upserts.
files, certificates, certificate_types, properties, tax_records, payments	Add File, Certificate, Property, Tax, and Payment APIs with file metadata, access control, and audit records.
municipal_service_directory, announcements	Add public content APIs for schemes, permits, helplines, downloadable documents, and citizen notices.
report_exports, system_settings, audit_logs, ai_audit_logs, ai_errors	Add Admin Reporting, Settings, Audit, AI Audit, and AI Error APIs with export tracking, non-secret settings, correlation IDs, and operational monitoring.

3. Route Structure

Routes are grouped by audience, access level, and service boundary. The implementation should keep route files small and mount them under a versioned API prefix.

```
backend/src/routes/
  v1/
    public.routes.js
    auth.routes.js
    citizen.routes.js
    staff.routes.js
    admin.routes.js
    ai.routes.js
    internal.routes.js
```

The initial route prefix is /api/v1. During transition, /api may be kept as an alias to /api/v1, but new frontend and service integrations should use /api/v1.

Public API

- GET /api/v1/public/announcements
- GET /api/v1/public/service-directory
- GET /api/v1/public/service-directory/:directoryNo
- GET /api/v1/public/file-tracking/:fileNo
- POST /api/v1/auth/register
- POST /api/v1/auth/login
- POST /api/v1/auth/password-reset/request
- POST /api/v1/auth/password-reset/confirm

Public file tracking must query only file_tracking records where public_tracking_enabled is true. It may return current public tracking fields and file_tracking_history.public_note; it must not return internal notes, officer-only comments, audit data, applicant private data, or storage keys.

Citizen API

- GET /api/v1/citizen/profile
- PATCH /api/v1/citizen/profile
- GET /api/v1/citizen/notification-preferences
- PATCH /api/v1/citizen/notification-preferences
- POST /api/v1/citizen/complaints
- GET /api/v1/citizen/complaints
- GET /api/v1/citizen/complaints/:complaintNo
- POST /api/v1/citizen/complaints/:complaintNo/feedback
- POST /api/v1/citizen/files
- GET /api/v1/citizen/files/:fileId/download
- POST /api/v1/citizen/chatbot/sessions
- POST /api/v1/citizen/chatbot/sessions/:sessionNo/messages
- POST /api/v1/citizen/chatbot/sessions/:sessionNo/escalate

Citizen APIs enforce ownership by user_id. Complaint submission must validate category, location, attachment file IDs, SLA mapping, and citizen address or ward information before creating complaints, complaint_status_history, optional file_tracking, and notification records.

Staff API

- GET /api/v1/staff/assignments
- PATCH /api/v1/staff/assignments/:assignmentId/status
- POST /api/v1/staff/complaints/:complaintNo/timeline
- POST /api/v1/staff/complaints/:complaintNo/proof-files
- GET /api/v1/staff/attendance/me
- POST /api/v1/staff/attendance/face-check
- POST /api/v1/staff/attendance/manual
- GET /api/v1/staff/presence
- GET /api/v1/staff/review-queue
- PATCH /api/v1/staff/review-queue/:reviewNo

Staff APIs enforce department membership and assignment scope through `department_members`, `complaint_assignments`, and RBAC permissions. Staff complaint timeline responses may include officer notes where permitted, but not internal audit notes unless the role explicitly allows them.

Admin API

- GET /api/v1/admin/users
- PATCH /api/v1/admin/users/:userId/status
- POST /api/v1/admin/users/:userId/roles
- GET /api/v1/admin/departments
- POST /api/v1/admin/departments
- GET /api/v1/admin/complaints
- POST /api/v1/admin/complaints/:complaintNo/assign
- GET /api/v1/admin/attendance
- GET /api/v1/admin/staff-presence
- GET /api/v1/admin/reports
- POST /api/v1/admin/reports/exports
- GET /api/v1/admin/reports/exports/:exportNo
- GET /api/v1/admin/audit-logs
- GET /api/v1/admin/ai-audit-logs
- GET /api/v1/admin/ai-errors
- GET /api/v1/admin/system-settings
- PATCH /api/v1/admin/system-settings/:settingKey

Admin APIs require explicit permissions, not only an `admin` role label. Report exports create `report_exports` records, write generated file metadata to `files`, and add audit entries before returning a downloadable result.

Knowledge Base and AI Admin API

- GET /api/v1/admin/knowledge-base
- POST /api/v1/admin/knowledge-base
- PATCH /api/v1/admin/knowledge-base/:kbNo
- POST /api/v1/admin/knowledge-base/:kbNo/publish
- POST /api/v1/admin/knowledge-base/:kbNo/archive
- POST /api/v1/admin/knowledge-base/:kbNo/embedding-jobs
- GET /api/v1/admin/human-review-queue
- PATCH /api/v1/admin/human-review-queue/:reviewNo

Only published knowledge_base records with `retrieval_allowed: true` and an `allowed_visibility` value may be used for RAG. Embedding jobs create or update `kb_embeddings`; Gemini responses must reference retrieved source IDs in chat message and audit metadata.

Internal Service API

- POST /api/v1/internal/ai/gemini/respond
- POST /api/v1/internal/ai/errors
- POST /api/v1/internal/face/enroll
- POST /api/v1/internal/face/verify
- POST /api/v1/internal/notifications/dispatch
- POST /api/v1/internal/jobs/chat-retention-cleanup
- POST /api/v1/internal/jobs/template-retention-cleanup

Internal routes require service authentication, network restrictions where available, request signing or mTLS in production, and correlation IDs. They are not exposed to browser clients.

4. Controller Structure

Controllers translate HTTP requests into service calls. They must not contain database queries, business decisions, AI prompt construction, face-matching logic, report generation logic, or authorization shortcuts.

```
backend/src/controllers/  
  auth.controller.js  
  public.controller.js  
  citizen.controller.js  
  complaint.controller.js  
  staff.controller.js  
  attendance.controller.js  
  admin.controller.js  
  knowledge.controller.js  
  aiGateway.controller.js  
  file.controller.js  
  report.controller.js  
  settings.controller.js
```

Controller responsibilities:

- read validated `req.params`, `req.query`, `req.body`, `req.user`, and `req.correlation_id`;

- call one service method per route action where practical;
- return standardized success responses;
- pass operational failures to the shared error handler;
- never return hidden fields such as storage keys, hashes, internal notes, encrypted embeddings, prompts, raw provider errors, or secrets.

5. Middleware Design

All non-health routes use the same middleware order:

1. `requestIdMiddleware` assigns or validates `correlation_id`.
2. `securityHeadersMiddleware` applies secure response headers.
3. `rateLimitMiddleware` applies endpoint-specific throttles.
4. `authMiddleware` validates JWT/session state where required.
5. `rbacMiddleware` checks role, permission, department scope, and object ownership.
6. `validationMiddleware` validates params, query, and body against request schemas.
7. `fileScanMiddleware` applies to upload endpoints before persistence.
8. Route controller calls service layer only.
9. Service layer writes database records and audit logs in the same logical workflow.
10. `errorHandler` returns a normalized error response and logs operational details.

6. Service Layer Design

Controllers must stay thin. Business rules belong in services, and MongoDB access belongs behind repositories or model-specific data access functions.

Service	Owns writes to
Auth Service	users, auth_sessions, password_reset_tokens, audit_logs
Complaint Service	complaints, complaint_assignments, complaint_status_history, feedback, notifications
File Service	files, file metadata, signed upload/download decisions
File Tracking Service	file_tracking, file_tracking_history
Directory Service	municipal_service_directory, announcements
AI Gateway Service	chatbot_sessions, chatbot_messages, ai_audit_logs, ai_errors, ai_human_review_queue
Knowledge Service	knowledge_base, kb_embeddings, source-file links
Attendance Service	attendance, attendance_summary, staff_presence, review queue links
Biometric Service	biometric_consent, face_templates, biometric audit events
Notification Service	notifications, user_notification_preferences
Reporting Service	report_exports, generated files, export audit entries
Settings Service	system_settings, settings-change audit entries

Service methods own transaction boundaries where MongoDB transactions are needed, idempotency decisions for retryable operations, audit log creation, downstream integration calls, and data redaction before controller responses.

7. Security Controls

- `complaints` remains the ticket collection. API names may use complaint or ticket wording in user-facing responses, but database writes must target `complaints`.
- Chatbot-created tickets must be created only through Complaint Service after AI tool-call validation. Gemini cannot write complaints directly.
- `staff_presence` is the read model for live presence dashboards. APIs must not scan raw attendance history to determine current presence.
- Face attendance requires active `biometric_consent`, an active `face_templates` record, liveness result, bounded confidence values from 0 to 1, and fallback handling.
- Raw face images must not be stored. Face APIs may process transient captures, then persist only approved attendance metadata and encrypted template references.
- Public directory and announcement endpoints return only `status: "published"` and public visibility records.
- `system_settings` may store non-secret settings or secret-manager reference IDs only. Plaintext API keys, JWT secrets, database credentials, and Gemini keys are forbidden.
- `files.storage_key`, `checksums`, internal notes, reviewer metadata, and audit links are never returned from public endpoints.
- AI errors and audit logs must carry `correlation_id` and redaction status.
- Use HTTPS only in deployed environments.
- Store JWT secrets, Gemini keys, MongoDB credentials, storage credentials, and encryption keys outside Git and MongoDB.
- Hash passwords with bcrypt or an approved equivalent.
- Store password reset tokens as hashes only.
- Apply request-size limits and upload type validation.
- Apply public endpoint rate limits for login, registration, password reset, chatbot, complaint submission, and file tracking.
- Enforce CORS allowlists for municipality-approved frontend origins.
- Log security-sensitive actions to `audit_logs` or `ai_audit_logs`.
- Block direct model writes to operational collections.

8. Authentication Flow

Authentication uses JWT access tokens backed by server-side session records in `auth_sessions`.

1. User submits login credentials to `/api/v1/auth/login`.
2. Auth Service verifies the user, password hash, account status, and lockout state.
3. Auth Service creates an `auth_sessions` record with device, IP hash, expiry, and revocation state.
4. API returns an access token and approved session metadata.
5. Protected requests send the access token in the approved authorization mechanism.
6. `authMiddleware` validates token signature, expiry, session ID, and `auth_sessions` status.
7. Logout or admin revocation marks the session revoked.
8. Refresh/session renewal must re-check `auth_sessions` before issuing a replacement access token.

Password reset flow:

1. `/api/v1/auth/password-reset/request` creates a hashed token in `password_reset_tokens`.
2. Raw reset tokens are sent only through the approved delivery channel.
3. `/api/v1/auth/password-reset/confirm` validates token hash, expiry, and unused status.
4. Password change updates the user password hash, marks the token used, revokes active sessions where policy requires it, and writes an audit log.

9. Authorization (RBAC)

The backend is the source of truth for access control. Frontend route guards are user-experience controls only.

- RBAC uses roles, permissions, and user-role assignment data.
- Department-scoped staff actions verify department_members.
- Object ownership is required for citizen profile, complaint, file, payment, certificate, and notification resources.
- Admin actions require named permissions such as complaint.assign, report.export, settings.update, knowledge.publish, and biometric.manage.
- Public routes still apply validation, rate limits, and response redaction.
- Internal routes require service identity and cannot rely on user JWTs alone.

10. API Standards

Request validators must mirror database validation requirements before Mongoose persistence:

- ObjectId fields must be valid before service execution.
- Enum values must match the database architecture.
- Confidence scores must be numbers from 0 to 1.
- Chatbot messages must include channel, language, session ID, and correlation ID.
- AI-generated responses must include response source, model name, prompt version, guardrail version, safety flag, escalation state, and retention expiry.
- Complaint submissions must include category, location where required, ward/address context, and attachment references owned by the requester.
- File tracking public updates must split public_note from internal_note.
- Password reset APIs store only token hashes and expiry metadata.
- Use JSON request and response bodies except for approved multipart file uploads.
- Use nouns for resource paths and action subpaths only where a workflow action is clearer, such as /publish, /archive, /assign, or /escalate.
- Use standard HTTP methods: GET for reads, POST for create/actions, PATCH for partial updates, and DELETE only where the database retention policy allows deletion.
- Use ISO 8601 timestamps in API responses.
- Use camelCase in JSON payloads unless a database field must be exposed by contract.
- Include correlation_id in every response meta or error object.
- Paginate list endpoints by default.

11. Error Handling Strategy

All API errors return this shape:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [],
    "correlation_id": "req_20260612_000001"
  }
}
```

Operational details, provider errors, stack traces, raw prompts, secrets, and biometric data must not be returned to clients. AI and integration errors are logged to ai_errors or audit_logs with redacted details.

Error categories:

HTTP status	Code pattern	Use case
400	VALIDATION_ERROR	Invalid body, params, query, enum, file type, or confidence score.
401	AUTHENTICATION_REQUIRED	Missing, expired, invalid, or revoked session token.
403	AUTHORIZATION_DENIED	Role, permission, ownership, department, or service-scope failure.
404	RESOURCE_NOT_FOUND	Missing resource or intentionally hidden resource.
409	CONFLICT	Duplicate unique fields, invalid state transition, or stale update.
429	RATE_LIMITED	Endpoint throttle exceeded.
500	INTERNAL_ERROR	Unexpected backend failure with redacted client message.
502/503	INTEGRATION_ERROR	Gemini, face service, storage, or queue dependency failure.

12. Request/Response Formats

Successful responses use this shape:

```
{
  "success": true,
  "data": {},
  "meta": {
    "correlation_id": "req_20260612_000001"
  }
}
```

List endpoints add pagination metadata: page, limit, total, has_next, and has_previous.

Create/update request example:

```
{
  "title": "Street light not working",
  "categoryId": "665f1b7a2f7c000000000001",
  "location": {
    "type": "Point",
    "coordinates": [76.003, 10.994]
  },
  "attachmentFileIds": ["665f1b7a2f7c000000000002"]
}
```

List response example:

```
{
  "success": true,
  "data": [],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 0,
    "has_next": false,
    "has_previous": false,
    "correlation_id": "req_20260612_000001"
  }
}
```

13. Integration Points

The API integrates with these internal and external systems through service-layer adapters:

Integration	API boundary	Data and control rules
MongoDB Atlas	Repository/model layer	All writes go through services; validators mirror collection rules; indexes support documented access patterns.
Gemini API	AI Gateway Service	Backend-mediated only; prompts, retrieval sources, safety decisions, and failures are logged with redaction.
Atlas Vector Search	Knowledge Service	Uses kb_embeddings for approved RAG retrieval; only active embeddings for published allowed content are used.
Face Recognition Service	Biometric and Attendance Services	Requires active consent; raw images are transient; stores encrypted template references and verification metadata only.
File/Object Storage	File Service	Stores metadata in files; clients receive signed access only after authorization.
Notification Provider	Notification Service	Creates notifications records before dispatch; failures are logged and retryable.
Report Generator	Reporting Service	Creates report_exports, generated file records, and audit logs for CSV/PDF exports.
Background Jobs	Internal Service API	Runs retention cleanup, embedding jobs, export cleanup, notification retries, and template deletion workflows.

14. API Versioning

- Current version: /api/v1.
- Breaking contract changes require a new prefix such as /api/v2.
- Non-breaking additions may remain in /api/v1, including optional response fields, new endpoints, and new enum values only where clients are designed to ignore unknown values.
- Deprecated endpoints must remain documented until all frontend, internal service, and municipality integrations are migrated.
- Versioning applies to public, citizen, staff, admin, AI, and internal service routes.
- API documentation and database migration notes must identify which API version first supports new collection fields or workflows.

15. Audit and Observability

The API must write audit or AI audit records for:

- login, logout, failed login, password reset, and session revocation;
- role, permission, department membership, and account-status changes;
- complaint creation, assignment, status update, resolution, and feedback lock;
- file upload, download authorization, public tracking update, and report export;
- knowledge-base publish/archive and embedding job creation;
- chatbot response generation, safety block, tool-call validation, escalation, and ticket creation;
- biometric consent grant/revocation, face template enrollment/deletion, attendance verification, failed liveness, and manual fallback;
- settings changes and secret-reference updates.

Every API request that mutates data must carry a correlation_id into audit logs, AI audit logs, AI errors, notifications, and any downstream service calls.

16. Implementation Order

1. Build shared middleware: correlation ID, auth, RBAC, validation, rate limit, and error handling.
2. Implement Auth, User, Role, Department, and Session APIs.
3. Implement File Service because complaints, profiles, directories, reports, and certificates depend on file metadata.
4. Implement Complaint, Assignment, Timeline, Feedback, and File Tracking APIs.
5. Implement Directory and Announcement APIs for public content.
6. Implement Attendance, Biometric Consent, Face Template, and Staff Presence APIs.
7. Implement AI Gateway, Chatbot Session, Knowledge Base, Embedding Job, AI Audit, and Human Review APIs.
8. Implement Admin Reporting, System Settings, and operational monitoring APIs.

17. Final Comparison Result

The database architecture is more complete than the previous API architecture. No database change is required from this comparison. The required change is to expand the API architecture so it explicitly supports all v4.0 database collections, validation rules, privacy boundaries, audit requirements, and AI/biometric workflows.

This document makes the API architecture consistent with the approved database architecture by defining route structure, controller structure, middleware design, authentication flow, authorization through RBAC, error handling strategy, API standards, request/response formats, security controls, service layer design, integration points, API versioning, module ownership, collection-specific rules, and implementation order.